# Practical - 9

## Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

**Problem Definition** Implement secure user registration by inserting user details into MySQL using MYSQLi. Validate input on frontend and backend, check duplicate email/username, hash passwords using password_hash(), and display appropriate messages.

## A. Theory

### 1. Is password_hash() used before storing passwords?

Yes. In process_register.php, before the INSERT query runs, the raw password is passed through password_hash($raw_password, PASSWORD_DEFAULT). The hashed version is then stored in the password_hash column — the plain text password is never saved to the database.

### 2. Is duplicate email/username checked?

Yes. Before inserting, a SELECT query runs on the students table:

SELECT * FROM students WHERE email = '$email' OR enrollment_id = '$enrollment'

If mysqli_num_rows() returns more than 0, the script stops and shows: "An account with this Email or Enrollment ID already exists!"

### 3. Are frontend and backend validations consistent?

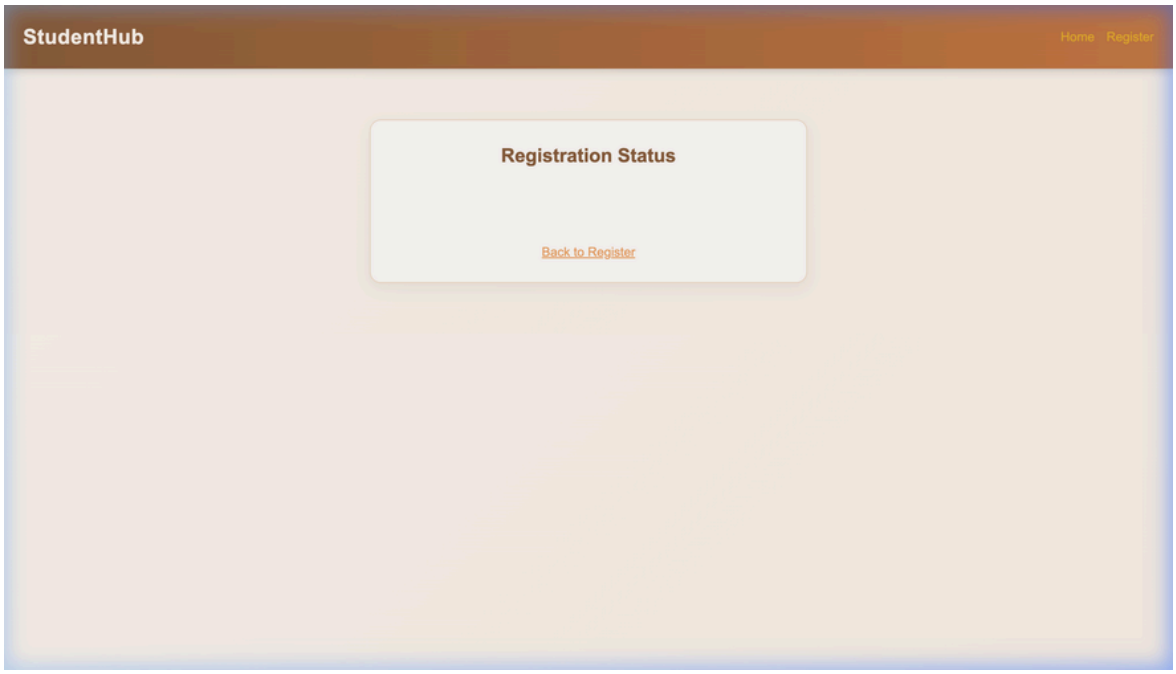
Yes. The HTML register.html uses required attributes, type="email" for the email field, and input constraints as frontend validation. The PHP backend then performs the same checks again — empty field check, email format validation, and duplicate check — ensuring both layers are aligned.

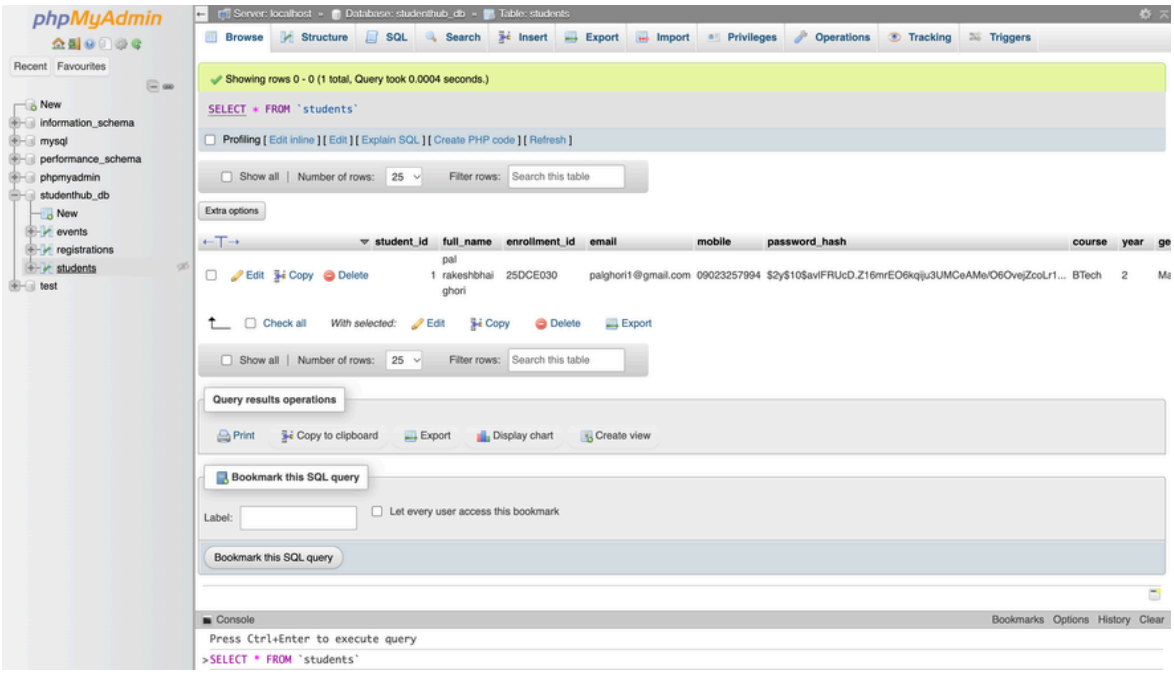
## B. Implementation

The implementation focused on secure user registration using MySQLi (basic procedural style):

| File | Role |
|---|---|
| db_connect.php | Connects to MySQL using mysqli_connect() |
| process_register.php | Sanitizes → Validates → Checks duplicate → Hashes password → Inserts into DB |
| register.html | Frontend form with basic HTML5 validation |

## C. Evidence of Work (Screenshots)




phpMyAdmin
Recent
Favourites
New
information_schema
mysql
performance_schema
phpmyadmin
studenthub_db
New
events
registrations
students
test
Server: localhost » Database: studenthub_db » Table: students
Browse
Structure
SQL
Search
Insert
Export
Import
Privileges
Operations
Tracking
Triggers
Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)
SELECT * FROM `students`
Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]
Show all | Number of rows: 25 Filter rows: Search this table
Extra options
student_id full_name enrollment_id email mobile password_hash course year
Edit Copy Delete 1 pal rakeshbhai ghori 25DCE030 palghori1@gmail.com 09023257994 $2y$10$avIFRUcD.Z16mrEO6kqjju3UMCeAMe/O6OvejZcoLr1... BTech 2
Check all With selected: Edit Copy Delete Export
Show all | Number of rows: 25 Filter rows: Search this table
Query results operations
Print Copy to clipboard Export Display chart Create view
Bookmark this SQL query
Label:
Let every user access this bookmark
Bookmark this SQL query
Console
Bookmarks Options History Clear
Press Ctrl+Enter to execute query
>SELECT * FROM `students`

## D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%209

## E. Challenges Faced:

MySQLi vs PDO: Understanding the difference between procedural mysqli_query() (Practical 9) and object-oriented PDO::prepare() (Practical 8). MySQLi is simpler but less flexible.
Duplicate Check Logic: Writing a single SELECT that checks both email AND enrollment_id in one query using OR.
Password Handling: Understanding why the raw password must be passed directly to password_hash() without mysqli_real_escape_string() first — escaping can corrupt the hash.

## F. Learning Outcomes:

MySQLi Procedural Connection — Learned how to use mysqli_connect(), mysqli_query(), and mysqli_num_rows() to interact with a MySQL database.
Server-Side Sanitization — Used mysqli_real_escape_string() to protect against basic SQL injection before building queries.
Secure Password Storage — Understood why plain text passwords must never be stored, and how password_hash() with PASSWORD_DEFAULT creates a secure one-way hash.
Duplicate Prevention — Implemented a pre-insert check using SELECT to prevent duplicate accounts before they cause database errors.
Form Validation (Two Layers) — Learned that frontend validation (HTML5) is for user experience, but backend validation is mandatory for security since frontend can be bypassed.

## Faculty Remarks: